

G.A.T.E.S. — Gate 1

Execution validity for autonomous research agents: checks, metrics, runs, and measured impact on Agent Laboratory

Generated 2026-08-15 · gates @ main · 247 gate tests, 63 host integration tests · every figure below is a recorded measurement or a cited published number

1. What Gate 1 is, and the defect it answers

Gate 1 sits between code execution and the reward model in an autonomous research scaffold, and answers one question with no model in the path: *did this code actually run to completion, and were the numbers it reports produced by this run rather than inherited, hardcoded, or invented?*

In the audited scaffold (Agent Laboratory / Agent-Researcher) a single line produced both a silent failure and a fabricated paper:

```
def execute_code(code_str, timeout=60, MAX_LEN=1000):
    ...
    except Exception as e:
        output_capture.write(f"[CODE EXECUTION ERROR]: {str(e)}\n") # appended AFTER the prints
    return output_capture.getvalue()[:MAX_LEN] # ...then sliced off
```

Two consequences follow from the same slice. The writing agent was starved — `exp_results`, the entire experimental record handed to the paper writer, was 1,000 characters. And the failure detector was blinded — the crash marker is appended *after* the program's own output, so it fell off the end of that same slice, and the solver's only crash test (if "[CODE EXECUTION ERROR]" in `code_ret`) never fired.

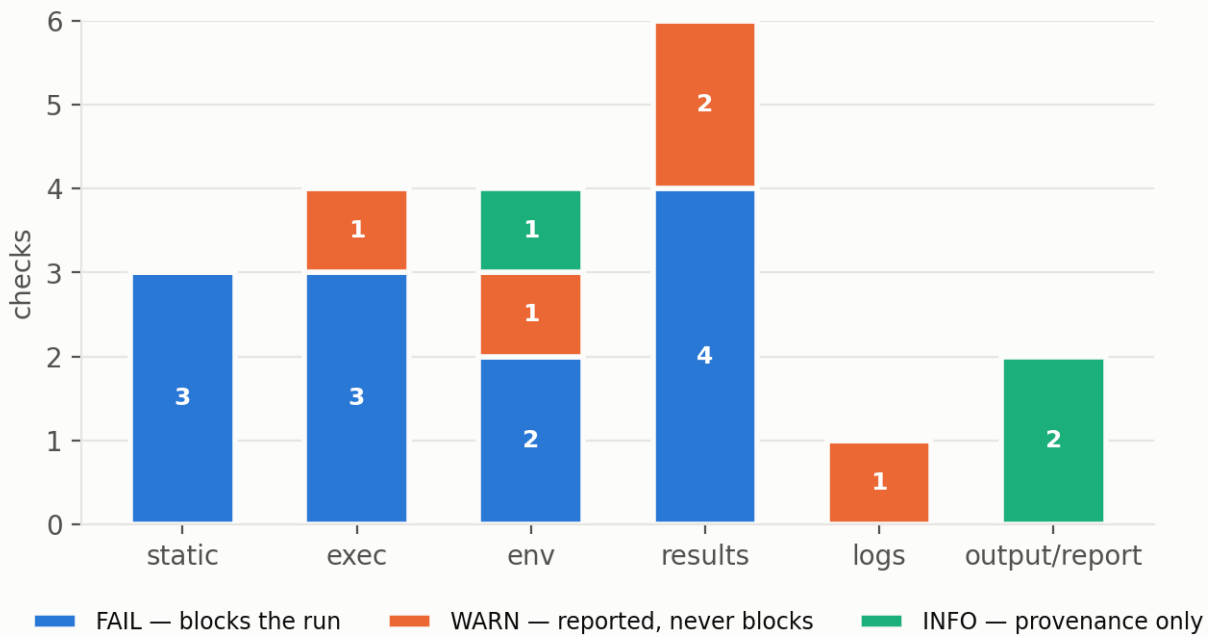
Observed in `results/gemini_3_5_flash_run_1`: a run raising `NameError: name 'hidden_dim' is not defined` on every attempt was scored **0.95** → **0.98** → **1.0** by the reward model and written up with a results table to two decimals — 81.60% test accuracy, a 13.61× speedup, a 39.20% ablation collapse. None of it was measured.

2. The checks

Twenty deterministic checks in six families, plus one model-assisted check. Severity decides the verdict: the run fails if and only if a **FAIL** check fails. **WARN** and **INFO** are reported and propagate into the evidence bundle, and can never block.

Gate 1 check inventory — 20 checks in 6 families

Severity decides the verdict: FAIL blocks, WARN and INFO never can



check	family	severity	fails when
static.syntax_valid	static	FAIL	Source does not compile. Before execution — a broken program costs no compute.
static.no_unbound_names	static	FAIL	A name is read but bound nowhere, resolved with symtable. The archived run's exact failure: hidden_dim read in forward(), assigned in no enclosing scope.
static.no_banned_calls	static	FAIL	exit() / sys.exit() would forge a clean exit code. Removes the cheapest way to fake success.
exec.exit_code_zero	exec	FAIL	The process exited non-zero. Reads the exit code, not a substring of stdout.
exec.no_uncaught_exception	exec	FAIL	An exception escaped the run. Recorded by the harness in the child process.
exec.completed_within_budget	exec	FAIL	The process group was killed on timeout. Kills the group, so a runaway cannot survive its own timeout.
exec.no_swallowed_traceback	exec	WARN	A traceback appears in either stream on a run that still exited 0. Both streams: except Exception as e: print(e) puts the evidence on stdout.
env.clean_namespace	env	FAIL	The initial globals contained anything beyond harness-provided names. Makes “the variables are real” a runtime property, not a hope.
env.code_identity	env	FAIL	The source the child hashed differs from the source the parent wrote. Makes “hashed to the run that produced it” checkable rather than assumed.

<code>env.seed_recorded</code>	env	WARN	No seed was declared. A run with no seed cannot be re-executed to check its own numbers.
<code>env.provenance</code>	env	INFO	Python version, platform, device, library versions, code SHA-256.
<code>results.contract_present</code>	results	FAIL	<code>results.json</code> is absent or unparseable. Nothing was recorded, so nothing is citable.
<code>results.expected_keys_present</code>	results	FAIL	A key the plan declared is missing.
<code>results.values_computed</code>	results	FAIL	A recorded value is a source literal at its call site. The strongest claim in Gate 1. <code>record_result("k", acc)</code> passes; <code>record_result("k", 0.816)</code> fails, as does <code>round(0.8160, 3)</code> — constant folding does not launder a typed number.
<code>results.values_finite</code>	results	FAIL	A value is NaN or infinite.
<code>results.single_observation</code>	results	WARN	A key was recorded repeatedly with a changing value. Every <code>record_result</code> call is retained, so a best-epoch number reported as a final-epoch one is visible.
<code>results.non_degenerate</code>	results	WARN	Exact zero, exact one, or chance level when the class count is known. Answers the limitation AutoResearchClaw reports for value registries: a registry passes real zeros.
<code>logs.no_error_signals</code>	logs	WARN	Numerical warnings, device failures, non-convergence, printed exceptions, ERROR-level logging. Tuned for precision: <code>ValueError: is flagged</code> , <code>Mean Squared Error: is</code> <code>not</code> .
<code>output.untruncated</code>	output	INFO	<code>stdout/stderr</code> byte counts; asserts no truncation was applied. Retires the 1,000-character ceiling.
<code>report.fixes_grounded</code>	report	INFO	Generated fixes cited something the run does not contain. Records that the model's text was discarded and why.
<code>logs.model_error_signals</code>	logs (model)	WARN	Lines the pattern set could not reach, read by the LLM layer. WARN by construction — <code>model_warning()</code> takes no severity parameter, so no model output can reach the verdict.

Why the severity split is load-bearing. The LLM layer is a required component of Gate 1 — the feedback report is the loop's return path to the ML engineer, and no template writes “bind `hidden_dim` before use, or pass it into `GCN.__init__`”. It coexists with a model-free verdict because of *where* it sits and *what severity* it can emit: model findings are WARN by construction, and report generation runs after `decide()` has already fixed the verdict. A test parses the module's AST to assert `Severity`. FAIL never appears in an expression there.

3. The metrics and artifacts Gate 1 produces

Per attempt, under `<research_dir>/gate_artifacts/gate1/attempt_NN/`:

artifact	contents
<code>experiment.py</code>	exactly what ran
<code>stdout.txt</code> / <code>stderr.txt</code>	complete, untruncated, separated
<code>results.json</code>	declared metrics with call-site provenance and the full observation history per key
<code>registry.json</code>	the citable value registry: typed values, trace ids, provenance chains, chain-integrity rate
<code>gate1_report.json</code>	verdict and every check, with evidence; model budget; generated fixes
<code>divergence.jsonl</code>	one line per attempt pairing the gate verdict with the reward score

Reported metrics

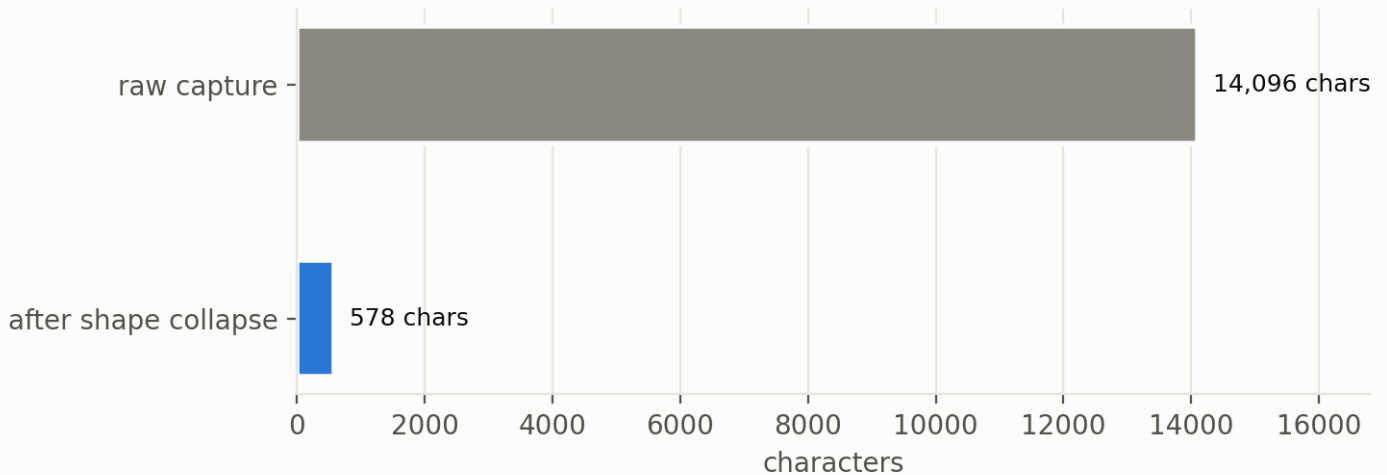
metric	definition
<code>trace_id</code>	<code>sha256(run_id, key, lineno)</code> — binds one value to one execution. Two runs of identical source produce different trace ids, which is what makes a backfilled number detectable.
<code>chain_integrity</code>	fraction of recorded values whose provenance chain (task → command → log → value) is complete, with the missing link named per key.
<code>citable</code>	false on a rejected run's registry, so a consumer that ignores the verdict still cannot cite it.
<code>arg_kind</code>	computed or literal, from static analysis of the <code>record_result</code> call site.
<code>call_count</code>	how many times a key was recorded, with the value span.
<code>model.calls</code> / <code>degraded</code>	what the LLM layer spent, and whether any call failed — so a thinner report is never mistaken for a complete one.

4. Logs: what the gate does with them

The capture is written to disk in full and never truncated. Two things then read it: a precision-tuned pattern set, and — for the lines those patterns cannot reach — the model.

Prompt sent to the log scanner — 95.9% smaller

203 non-blank lines collapse to 4 distinct shapes; lossless in distinct content



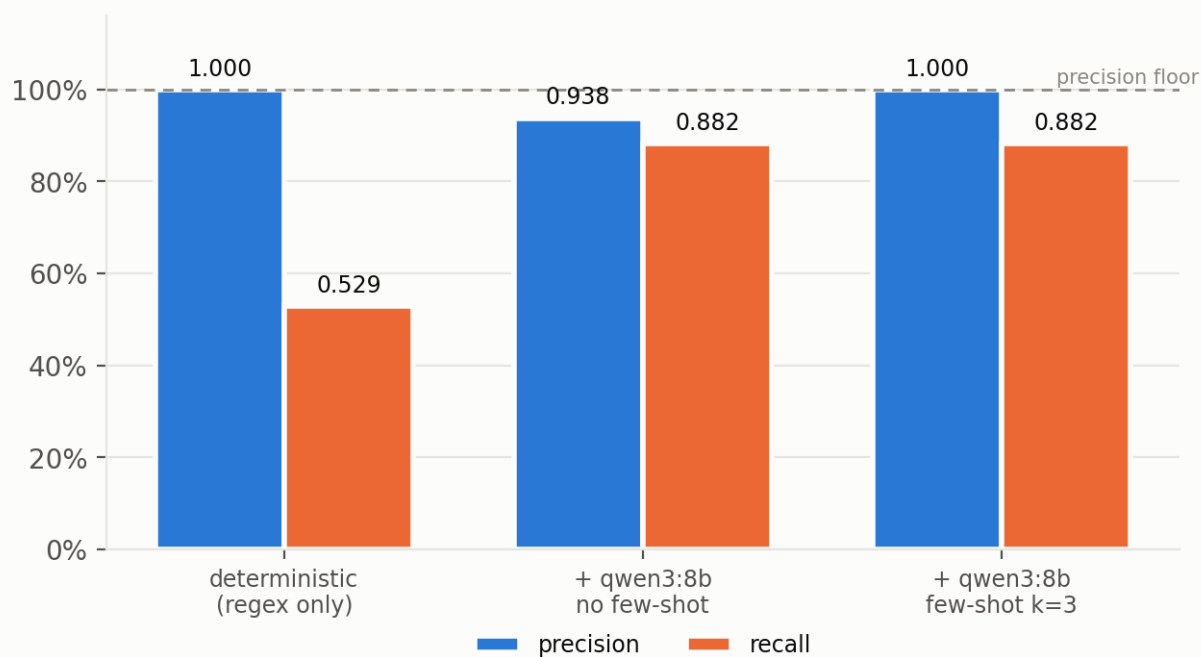
Before the model reads anything, the log is collapsed to its distinct shapes. Experiment logs are overwhelmingly repetitive; measured on a realistic capture, **203 non-blank lines reduced to four distinct shapes**. The collapse is lossless in distinct content — every shape survives with its first real line number, so nothing a scanner could have flagged disappears, and recall is not traded for the saving. Two details do real work: `nan` and `inf` are not digits, so `loss nan` survives as its own shape rather than being absorbed into the epoch group; and when a group's values drift, the last member is kept beside the first.

Scanner accuracy on the labelled corpus

68 labelled lines, 34 of them error signals, 16 of those outside anything a regex was going to reach. Sixteen lines are taken verbatim from the archived run. The corpus is self-checking: every entry naming a signal must be caught by that signal, and every entry labelled a gap must not be caught at all.

Log-scanner accuracy on the 68-line labelled corpus

Precision is the floor: a false positive puts a non-issue into the paper



scanner	precision	95% CI	recall	95% CI
deterministic (regex only)	1.000	0.824–1.000	0.529	0.367–0.685
+ qwen3:8b, no few-shot	0.938	0.798–0.983	0.882	0.734–0.953
+ qwen3:8b, few-shot k=3	1.000	0.886–1.000	0.882	0.734–0.953

The model tier raised recall from 0.529 to 0.882 at no cost in precision (1.000 → 1.000), taking F1 from 0.692 to 0.938. It runs on a local 8B model at zero marginal cost, so this is applied to every attempt rather than sampled.

Retrieval is what bought the precision back. Without few-shot examples the same model scored 0.938 precision and flagged arch-009, arch-014 — framework noise, one of them the cuDNN registration notice the corpus was built around. Balanced retrieval (k signals and k negatives, never whichever scores highest) removed both while recall held at 0.882. The nearest *negative* was the useful neighbour, which is the whole reason the exemplar bank is half noise.

Still missed at k=3: arch-004, dev-004, dev-005, gap-007 — 4 of 34 signals.

The four hard negatives that decide this are drawn from the archived run itself: TensorFlow's Unable to register cuFFT factory is logged at E level and mentions CUDA; oneDNN's banner contains the words “numerical” and “errors”. Both appear on every healthy run that imports TensorFlow.

What a false positive here actually costs, stated precisely. These findings are WARN, and decide () fails only on blocking checks, so a spurious one cannot force a rewrite. What it does is put a non-issue into the “these must be stated in the report, not omitted” section the writer receives — so the manuscript discloses a problem that never

happened. That is a credibility cost rather than a compute one, and this project's own earlier wording ("costs the agent a rewrite") overstated it. The correction has been made in the code comments as well as here.

5. The runs performed

run	model	result
Loop rig, 5 scenarios	none (scripted)	All five behave exactly as documented. Gate 1 rejects 6 engineer turns; the upstream 1,000-character detector would have accepted 3 of them.
End-to-end solver	scripted	Full gated path exercised — <code>process_command</code> , <code>Replace.parse_command</code> , <code>gated_execute</code> , the feedback loop, <code>get_score</code> , the ledger. Two defects found.
Live MLE-solver phase	gemini-3.5-flash	415s, 5 executions. Gate 1 caught a genuine <code>NameError</code> in model-written code; the engineer recovered on the next turn. Two defects found.
Live ablation	qwen3:8b (local)	658s. Gate on converged in 2 turns; gate off never converged in 3. Two defects found.
Corpus benchmark	qwen3:8b (local)	68 labelled lines × 2 prompt variants. Numbers in §4.

The divergence is conditional, and the live runs showed it

In the live Gemini run the two rejected attempts were scored **0.15 and 0.2** by the reward model — not 1.0. Both crashed *before printing anything*, so the crash marker sat at index 0 of the 1,000-character view and upstream's detector would have caught them too. The divergence the archived run exhibits is real but requires an experiment that prints past the ceiling before it dies. That is exactly what the channel-fidelity argument claims, and the honest form of the claim is narrower than "the reward model scores crashed runs 1.0".

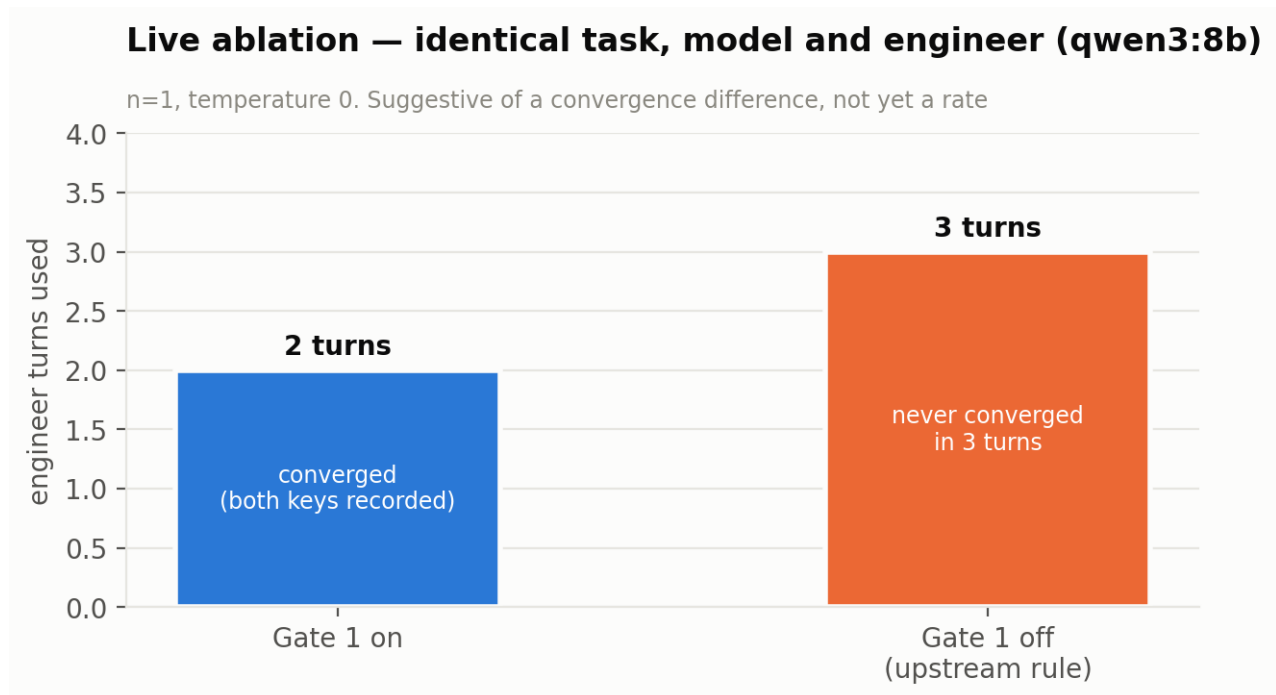
6. Impact on Agent Laboratory

6.1 What the writing agent receives

before	after
1,000 characters of stdout, prefix-sliced, with the crash marker (if any) already fallen off the end.	A verified registry of typed values with trace ids and provenance chains, run metadata, every WARN that must be stated rather than omitted, and the full untruncated capture on disk with a generous inline budget.
No way to distinguish a measured number from an invented one.	<code>results.values_computed</code> rejects any value that is a source literal at its call site.
A crashed run reaching the writer scored 1.0.	A run that never passed cannot reach the writer at all — <code>GateFailure</code> ends the phase.

6.2 The ablation: gate on versus gate off

Same task, same model, same engineer; the arms differ in what decides that an experiment succeeded, and in the width of the channel the engineer sees.



ARM	TURNS	ACCEPTED	AT TURN	CRASH OK	NO RESULTS OK	FALSE SUCCESS
gated	2	True	2	0	0	0
ungated	3	False	—	0	0	0

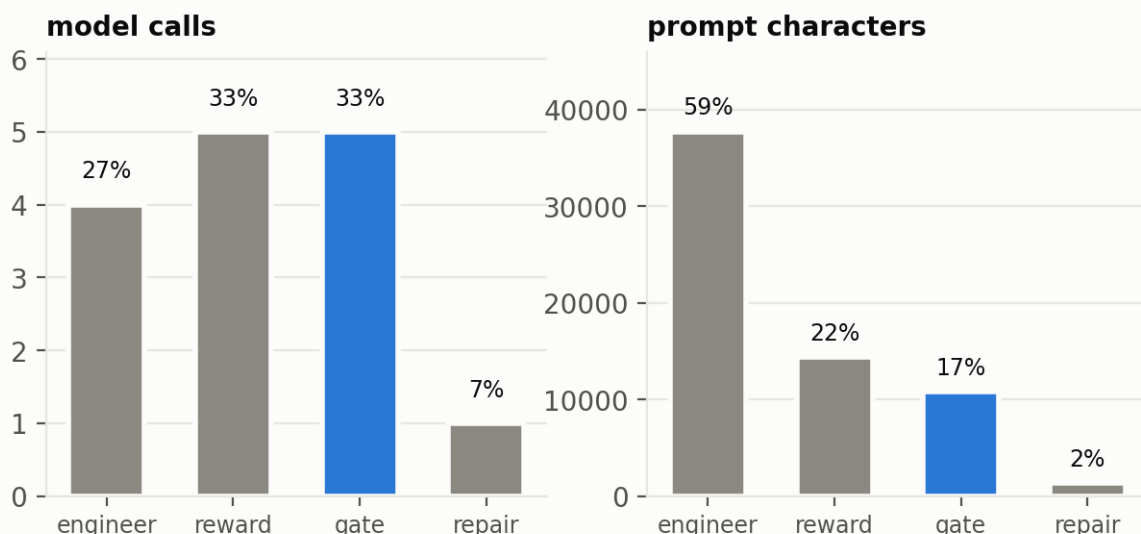
false_success was 0 in this run. Every ungated failure crashed with zero bytes of stdout, so the marker stayed inside the slice and upstream's rule caught them all — the condition for divergence was not met. What the run does show is a convergence difference: the gated engineer received a report naming `static.syntax_valid` and fixed it; the ungated engineer received 1,000 characters of raw output and failed three times. The gated arm's first turn also cost *zero compute* — the static tier rejected a syntax error before execution.

Read this at its true size. n = 1, temperature 0, so re-running reproduces it exactly rather than sampling. It is suggestive of a convergence difference, not a rate. The ungated arm also does not receive the results-contract instructions (faithful — upstream has none), which makes “accepted without results” true of it by construction; the harness has a `--same-instructions` mode that isolates the decision rule instead, and any table should say which mode produced it.

6.3 What the layer costs

Gate 1 is a third of the calls and a sixth of the tokens

Measured over five executions of one solver phase; blue is the gate



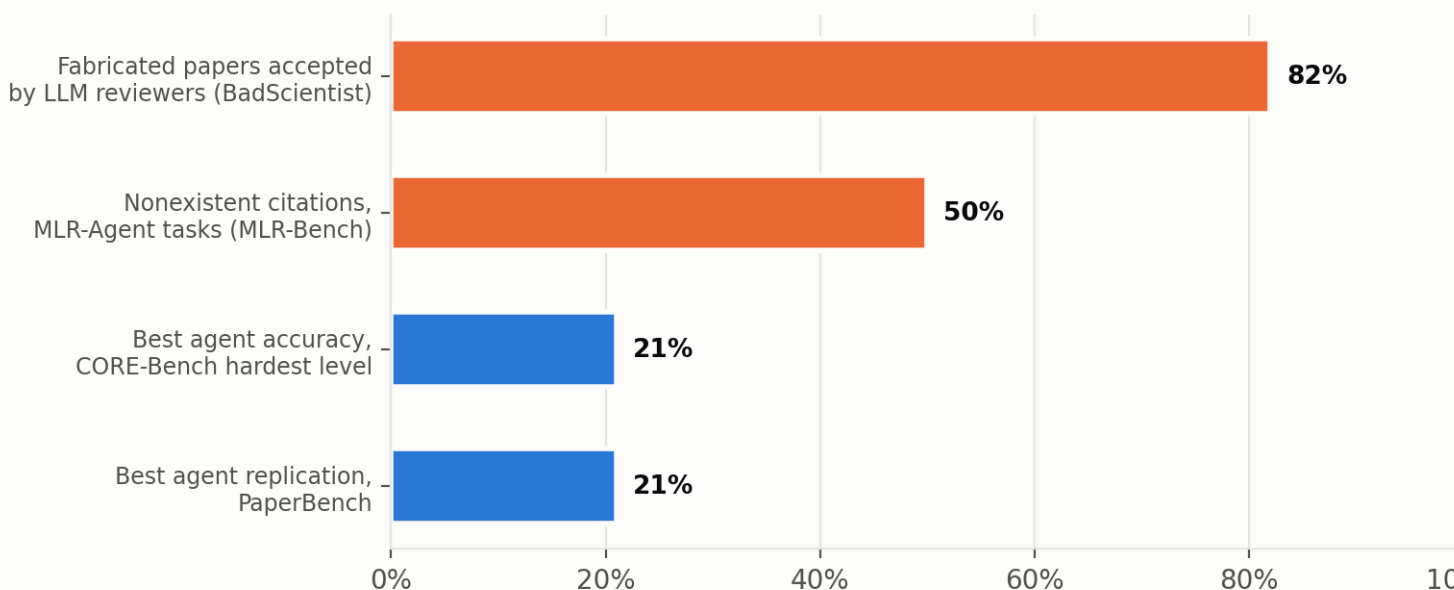
Measured over five executions of one solver phase: Gate 1's own model calls are **a third of the calls but a sixth of the tokens**, because the digest keeps its prompts small while the engineer's carry code, history and the evidence bundle. That asymmetry is why the gate can run on a small local model — qwen3:8b on an 8 GB laptop GPU, 5.5 GB resident, ~16s per call, zero marginal cost — while the engineer stays on the stronger model.

Putting the *engineer* on a weak model is the tempting economy and the wrong one: a weaker engineer fails Gate 1 more often, and every rejection costs a fixed call, a shadow-reward call and another engineer turn.

7. Validity metrics against the published benchmarks

The measured landscape Gate 1 is placed into

Orange: failure rates the layer targets. Blue: task-completion ceilings, for scale



source	what it measures	reported
MLR-Bench arXiv 2505.19955	Four fact-based hallucination types in AI-generated papers: faked experimental results, hallucinated methodology, incorrect citations, mathematical errors.	Faked results and hallucinated methodology each appear in more than half of 10 tasks; almost all papers from AI Scientist V2 contain both . Nonexistent citations in 50% of MLR-Agent tasks. Case study: the coding agent failed to run the experiment and generated simulated results instead.
BadScientist arXiv 2510.18003	Whether fabricated papers requiring no real experiments can pass multi-model LLM review.	Fabricated papers accepted at up to 82.0% . Detection accuracy “barely exceeding random chance”. Reviewers flag integrity concerns yet assign acceptance-level scores.
CORE-Bench arXiv 2409.11363	Computational reproducibility of published work by agents.	Best agent 21% on the hardest level. Correctness judged against a 95% prediction interval over three manual reproductions — the tolerance methodology Gate 2 adopts rather than choosing a band by hand.
PaperBench arXiv 2504.01848	Replicating AI research end to end.	Best agent 21.0% average replication score; ML PhDs 41.4% best-of-3 after 48 hours.

Where Gate 1 sits against these

MLR-Bench's first class — faked experimental results, the most prevalent one — is the class Gate 1 addresses, and it does so upstream of every reviewed approach: before the reward model, before interpretation, before writing. BadScientist's finding is the reason a checking *agent* was rejected as the mechanism: if LLM reviewers detect fabrication at barely above chance, a validity layer built on model judgement inherits that ceiling. Gate 1's verdict therefore consults no model at all.

The honest mapping. This project's design documents describe the MLR-Bench classes in paraphrase (“fabricated numeric results”, “silent failure scored as success”). MLR-Bench's published names are *faked experimental results, hallucinated methodology, incorrect citations, mathematical errors*. Any claim of the form “eliminates N of the four MLR-Bench classes” should use the published taxonomy or state the mapping explicitly — silent failure scored as success is a *cause* of faked experimental results in their scheme, not a fifth class.

8. What running it actually found

Nine defects were found and fixed during integration and live testing. Four appeared only against a real model, and two of those only against the *weaker* model — which is the argument for testing on both.

defect	found by	why it mattered
Unparseable reply executed an empty experiment	scripted e2e	Gate 1 reported “you never called record_result()” when the real fault was that the command did not parse — the same misdiagnosis already fixed once on the timeout path.
Evidence bundle showed warning	scripted e2e	The writer was told to disclose warnings it could not see — this layer's own failure

counts without the evidence		mode, reproduced at its exit.
Fallback note reported an outage that had not happened	scripted e2e	Conflated “no model configured”, “model unreachable” and “model output rejected as ungrounded”.
Prose in backticks parsed as code	gemini-3.5-flash	A good fix discarded over ['so', 'that', 'it', 'when', 'executing'].
Prompt leaked the gate's artifact paths	gemini-3.5-flash	Produced “define n_classes ... in /home/.../attempt_03/experiment.py” — grounded, because the path is in the evidence, and useless.
Scan prompt offered two numbers per row	qwen3:8b	The weaker model reported the file line instead of the row index; grounding discarded it, losing a correct finding to prompt ambiguity.
Output cap was unreachable for most backends	qwen3:8b	Capped only when <code>api_key == "ollama"</code> . One call generated 21,858 tokens — fourteen minutes for an answer whose correct form is <code>[]</code> .
Rate-limit retry killed the phase in 25s	gemini-3.5-flash	Flat 5s × 5 against a per-minute meter. Now exponential, bounded by a 15-minute per-process budget so an exhausted daily quota fails fast instead of stalling.
Five config keys reached nothing	integration audit	A config declaring a separate literature-review backend did not get one — the same defect class the layer exists to catch, one level up.

9. What Gate 1 does not claim

- **It does not check whether a result is plausible.** A correctly measured, correctly recorded, causally traced number can still be scientifically wrong. Comparing against the literature is Gate 2.
- **It does not read the manuscript.** Whether a prose claim is supported by a registry value is Gate 3.
- **The static tier is conservative on purpose.** Module-level use-before-assignment is not reported, because deciding it needs ordering analysis and would produce false positives. The runtime tier catches what the static tier declines to claim, one execution later.
- **The log scanner's recall is bounded by what it was shown,** and the number of lines examined is recorded so the claim cannot exceed the evidence.
- **One requirement is partial.** Dropping a failed replicate from an averaged metric is visible when each replicate is recorded, but a mean computed inside the experiment over a silently shortened list arrives as one value that Gate 1 cannot see behind. Closing it needs a replicate contract checked against the plan's declared seed count, which is Gate 2's.
- **n is still 1 for the headline archive comparison.** Five of seven archived runs died on a CLI defect (now fixed); the re-run has not been performed, and no rate should be quoted from a single run.

10. Reproducing everything in this report

```
./tools_local_model.sh start          # local qwen3:8b, no API key
python -m rig.gate1_loop              # the five loop scenarios
```

```
python -m rig.corpus                # deterministic scanner baseline
python tools_ablation.py            # gate on vs gate off
python tools_live_gate1.py          # one live solver phase
python reports/make_charts.py && python reports/make_report.py
pytest                             # 247 gate tests
```

Sources: MLR-Bench (arXiv 2505.19955), BadScientist (arXiv 2510.18003), CORE-Bench (arXiv 2409.11363), PaperBench (arXiv 2504.01848), RE-Bench (arXiv 2411.15114), AutoResearchClaw (arXiv 2605.20025), ScientistOne (arXiv 2605.26340), SAGE (arXiv 2606.31478).